Intelligent Information Systems
Security Engineering

Performance Evaluation of NIST Post-Quantum ML-KEM (FIPS 203) in TLS for ARM64 Devices

Submitted by:

Muhammad Nour Ali

Supervised by:

Dr. Christine Zenieh - Eng. Ahmad Shekha

2026

1 Contents

2 List of Tables and Figures 3

3 Table of Abbreviations 4

4 Abstract 5

5 Introduction 7

5.1 Research Problem 7

5.2 Objective of the Project 8

6 Theoretical Study and Literature Review 9

6.1 Preface 10

6.2 Classical Public-Key Cryptography and the Quantum Threat 10

6.2.1 Classical Public-Key Assumptions 10

6.2.2 Shor's Algorithm and Cryptographically Relevant Quantum Computers 10

6.2.3 Post-Quantum Cryptography (PQC) 10

6.2.4 Overview of the ML-KEM Scheme 11

6.2.5 Transport Layer Security (TLS) 1.3 13

6.2.6 Integrating ML-KEM into TLS 1.3 15

6.2.7 KEM vs. Diffie-Hellman 15

6.2.8 Security Properties of PQC-TLS 15

6.2.9 Summary of Theoretical Foundations 15

6.3 Literature Review 16

6.3.1 Critical Analysis and Research Gap 17

7 Proposed Post-Quantum TLS 1.3 Key-Establishment Architecture 19

7.1 Preface 20

7.2 Design Objectives 20

7.3 Integration Strategy 20

7.4 Benchmarking Architectural Decisions 22

7.5 Summary 23

8 Implementation 24

8.1 Preface 25

8.2 Environment Overview 25
8.3 Software Stack 25
8.4 System Components 25
8.5 Initialization and Configuration 26
8.6 System Execution Workflow 27
8.7 Conclusion 28
9 Results and Analysis 29
9.1 Introduction 30
9.2 Raw Primitive Results 30
9.2.1 Client Side 30
9.2.2 Server Side 32
9.2.3 Why ML-KEM Outperforms X25519 on ARM64 34
9.3 TLS Integration Results 34
9.3.1 Client Side 34
9.3.2 Server Side 35
9.3.3 Wall-time 37
10 Conclusion 38
11 References 39

2 List of Tables and Figures

Table 1 Sizes (in bytes) of keys and ciphertexts of ML-KEM 12
Table 2 Decapsulation failure rates for ML-KEM 13
Table 3 Summary of reviewed papers 16
Table 4 liboqs building tools 27

Figure 1 A simple view of key establishment using KEM 12
Figure 2 TLS 1.3 Message Flow (based on RFC 8446). 14
Figure 3 Raw ML-KEM (Native Direction) 21
Figure 4 TLS-MLKEM (Reversed Direction) 22
Figure 5 Raw ML-KEM Encaps Vs X25519 Derive 31
Figure 6 KeyGen, ML-KEM, X25519 server and client 33
Figure 7 ML-KEM Decaps Vs X25519 Derive 33
Figure 8 TLS CPU time ML-KEM Vs X25519 34
Figure 9 Message size overhead, client side 35
Figure 10 TLS Server CPU Time ML-KEM Vs X25519 36
Figure 11 Message size overhead, server side 37

3 Table of Abbreviations

Abbreviation	Meaning
ARM	Advanced RISC Machine
AVX2	Advanced Vector Extensions 2
CSV	Comma-Separated Values
DH	Diffie-Hellman
DLP	Discrete Logarithm Problem
EC	Elliptic Curve
ECDH	Elliptic Curve Diffie-Hellman
FIPS	Federal Information Processing Standards
IFP	Integer Factorization Problem
IND-CCA	Indistinguishability under Adaptive Chosen Ciphertext Attack
IoT	Internet of Things
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
ML-KEM	Module-Lattice-Based Key Encapsulation Mechanism
MLWE	Module Learning With Errors
NIST	National Institute of Standards and Technology
OQS	Open Quantum Safe
PQC	Post-Quantum Cryptography
PSK	Pre-Shared Key
QKD	Quantum Key Distribution
RFC	Request for Comments
RSA	Rivest-Shamir-Adleman
SIMD	Single Instruction Multiple Data
SL-DSA	Stateless Hash-Based Digital Signature Algorithm
TEE	Trusted Execution Environment
TLS	Transport Layer Security
UEFI	Unified Extensible Firmware Interface

4 Abstract

The emergence of cryptographically relevant quantum computers threatens the security of classical public-key algorithms such as RSA and elliptic-curve Diffie-Hellman, prompting an accelerated transition toward post-quantum cryptography (PQC).

Recent standardization efforts by NIST, particularly the adoption of ML-KEM (FIPS 203) for key establishment, have positioned lattice-based KEMs as the foundation of future secure communication protocols. However, integrating PQC into Transport Layer Security (TLS) introduces non-trivial performance and bandwidth considerations that must be empirically characterized before large-scale deployment.

In this work, we implement and benchmark a complete TLS 1.3 stack using ML-KEM-768 as the key-exchange mechanism and compare it against the widely deployed X25519 ECDH primitive.

Our evaluation spans raw cryptographic operations (key generation, encapsulation, decapsulation), full TLS handshakes on both client and server, CPU-time distributions, latency, and message-size overhead.

The results show that ML-KEM-768 achieves sub-millisecond encapsulation and decapsulation with tightly concentrated distributions, and server-side steady-state timings remain consistently below 2 ms. When integrated into TLS 1.3, ML-KEM introduces only a modest increase in handshake latency relative to X25519, with both client- and server-side timings clustering in the 15–30 ms range. CPU-time overheads remain similarly bounded.

The primary cost of PQC adoption arises from message sizes: ML-KEM handshake messages are significantly larger than those of X25519, reflecting the structural requirements of lattice-based cryptography and aligning with observations in prior PQC-TLS studies.

Overall, our findings confirm that ML-KEM-768 provides quantum-resistant key establishment with performance characteristics compatible with real-world TLS deployments. While bandwidth overhead remains the dominant challenge, the computational cost of PQC-TLS is already competitive with - and in some cases more stable than - classical elliptic-curve configurations.

These results reinforce the feasibility of transitioning to quantum-safe TLS in latency-sensitive, resource-constraint environments and contribute empirical evidence toward the broader migration roadmap envisioned by current PQC standards and research.

الملخص

إن ظهور الحواسيب الكمومية المتصلة بعلم التشفير يهدد أمن خوارزميات المفاتيح العامة الكلاسيكية مثل RSA و–Diffie Hellman المبنية على elliptic-curve، مما يدفع نحو انتقال متسارع إلى التشفير ما بعد الكم (PQC) وقد وضعت جهود القياس الحديثة التي يقودها المعهد الوطني للمعايير والتقنية(NIST) ، ولا سيما اعتماد ML-KEM (FIPS 203) لتأسيس المفاتيح، آليات التغليف القائمة على الشبكات (lattice-based KEMs) كأساس لبروتوكولات الاتصال الآمنة المستقبلية. ومع ذلك، فإن دمج التشفير ما بعد الكم في بروتوكول أمان طبقة النقل (TLS) يفرض اعتبارات غير بسيطة تتعلق بالأداء وعرض الحزمة، والتي يجب توصيفها تجريبيًا قبل أي نشر واسع النطاق.

في هذا العمل، قمنا بتنفيذ وقياس أداء حزمة TLS 1.3 كاملة باستخدام ML-KEM-768 كآلية لتبادل المفاتيح، ومقارنتها بالبديل واسع الانتشار X25519 المبني على ECDH .

يشمل تقييمنا العمليات التشفيرية الخام (توليد المفاتيح، التغليف، فك التغليف)، ومصافحات TLS الكاملة على كل من العميل والخادم، وتوزيعات زمن المعالج(CPU time) ،وحجم الرسائل المتبادلة.

تُظهر النتائج أن ML-KEM-768 يحقق تغليفًا وفك تغليف دون مستوى الميلي ثانية مع توزيعات شديدة التركّز، وأن أزمنة الخادم في الحالة المستقرة تبقى باستمرار أقل من 2 ميلي ثانية. وعند دمجه داخل TLS 1.3 ، يضيف ML-KEM زيادة طفيفة فقط في زمن المصافحة مقارنة بـX25519 ، حيث تتجمع أزمنة العميل والخادم معًا ضمن نطاق 15–30 ميلي ثانية. كما تبقى زيادات زمن المعالج ضمن حدود مشابهة.

وتتمثل الكلفة الأساسية لاعتماد التشفير ما بعد الكم في أحجام الرسائل: إذ تكون رسائل مصافحة ML-KEM أكبر بكثير من نظيراتها فيX25519 ، مما يعكس المتطلبات البنيوية للتشفير القائم على lattice، ويتماشى مع الملاحظات الواردة في دراسات سابقة حول TLS ما بعد الكم.

بشكل عام، تؤكد نتائجنا أن ML-KEM-768 يوفر تأسيس مفاتيح مقاومًا للكم مع خصائص أداء متوافقة مع عمليات نشر TLS في العالم الحقيقي. وبينما يبقى عبء عرض الحزمة هو التحدي الأكبر، فإن الكلفة الحسابية لـ TLS ما بعد الكم أصبحت بالفعل منافسة — وفي بعض الحالات أكثر استقرارًا — من التكوينات الكلاسيكية المبنية على elliptic-curve .

تدعم هذه النتائج إمكانية الانتقال إلى TLS آمن كموميًا في البيئات الحساسة للزمن ذات الموارد المحدودة، وتقدم دليلًا تجريبيًا يساهم في خارطة الطريق الأوسع للانتقال نحو التشفير ما بعد الكم كما تبرزها المعايير والأبحاث الحالية.

5 Introduction

The rapid advancement of quantum computing has introduced a fundamental threat to the cryptographic foundations that secure today's Internet. Classical public-key algorithms—most notably RSA and elliptic-curve Diffie–Hellman (ECDH)—derive their security from mathematical problems that are believed to be intractable for classical computers. However, the emergence of cryptographically relevant quantum computers capable of executing Shor's algorithm would render these schemes breakable in polynomial time, compromising confidentiality, integrity, and authentication across global communication systems.

This looming risk has accelerated the transition toward post-quantum cryptography (PQC), a class of algorithms designed to remain secure even in the presence of large-scale quantum adversaries.

In response to this threat, the U.S. National Institute of Standards and Technology (NIST) initiated a multi-year standardization effort to identify quantum-resistant public-key primitives. After extensive cryptanalysis and evaluation, NIST selected ML-KEM—a module-lattice-based key-encapsulation mechanism derived from CRYSTALS-Kyber—as the official standard for key establishment (FIPS 203). ML-KEM offers strong security guarantees rooted in the hardness of the Module Learning With Errors (MLWE) problem and provides parameter sets (512, 768, 1024) that balance performance and security for real-world deployment. Its selection positions lattice-based KEMs as the foundation of future secure communication protocols.

At the same time, the Transport Layer Security (TLS) 1.3 protocol remains the dominant mechanism for securing Internet traffic. TLS 1.3 relies on ephemeral Diffie–Hellman key exchange—typically X25519—for forward secrecy and authenticated key establishment. Integrating PQC into TLS is therefore a critical step toward achieving a quantum-safe web. However, PQC primitives differ substantially from classical elliptic-curve algorithms in terms of computational cost, key sizes, ciphertext sizes, and message structure. As highlighted in prior studies, these differences introduce non-trivial performance and bandwidth implications that must be empirically evaluated before PQC can be deployed at scale.

5.1 Research Problem

While ML-KEM has been standardized and is widely regarded as efficient among PQC candidates, its practical impact on TLS 1.3 handshake performance—including latency, CPU cost, and message size overhead—remains insufficiently characterized in controlled, reproducible environments. Existing literature provides valuable insights but leaves several gaps:

- [6] a PQC-TLS can achieve competitive handshake throughput but emphasize significant transmission overhead due to large PQC key shares and certificates.

- [4] show that PQC-only TLS can outperform classical TLS in some scenarios, yet hybrid PQC-QKD approaches introduce substantial overhead.
- Studies on constrained environments (e.g., ARM TEE [5]) confirm that ML-KEM is computationally efficient, but they do not evaluate full TLS handshakes.
- Prior PQC-TLS experiments often rely on hybrid modes, simulated networks, or partial implementations rather than complete TLS 1.3 stacks.

Thus, there is a clear need for fine-grained, operation-level benchmarking of ML-KEM within a full TLS 1.3 implementation on ARM64 machines, directly compared against the widely deployed X25519 primitive.

5.2 Objective of the Project

This project aims to provide a comprehensive, empirical evaluation of ML-KEM-768 as a key-exchange mechanism within TLS 1.3. The objectives are:

- Implement a complete TLS 1.3 stack
- Benchmark raw cryptographic operations
- Measure full TLS 1.3 handshake performance
- Quantify message size overhead
- Compare ML-KEM-768 directly with X25519
- Interpret results in the context of existing PQC-TLS research

6 Theoretical Study and Literature Review

6.1 Preface

The following establishes the formal foundations required to analyze and implement quantum-resistant key establishment within TLS 1.3. By presenting the essential mathematical structures underlying lattice-based constructions, the operational semantics of ML-KEM as standardized in FIPS 203, and the handshake architecture of TLS 1.3 as specified in RFC 8446, this chapter provides the conceptual and technical scaffolding upon which the subsequent experimental evaluation is built. The material collected here is not intended as a historical survey nor as a discussion of related work, but rather as a coherent reference framework that clarifies the assumptions, algorithms, and protocol behaviors that govern the transition from classical to post-quantum secure communication.

6.2 Classical Public-Key Cryptography and the Quantum Threat

6.2.1 Classical Public-Key Assumptions

Traditional public-key cryptography relies on mathematical problems believed to be hard for classical computers:

- Integer Factorization Problem (IFP) — basis of RSA. *[4]*
- Discrete Logarithm Problem (DLP) — basis of Diffie–Hellman and elliptic-curve Diffie–Hellman (ECDH). *[3]*

6.2.2 Shor's Algorithm and Cryptographically Relevant Quantum Computers

Shor's algorithm solves both IFP and DLP in polynomial time, rendering RSA and ECDH insecure once large-scale quantum computers exist *[4]*. This motivates the transition to post-quantum cryptography (PQC).

6.2.3 Post-Quantum Cryptography (PQC)

Definition:

PQC refers to asymmetric cryptographic algorithms designed to remain secure even against adversaries equipped with quantum computers. *[6]*

NIST launched the PQC standardization process in 2016, evaluating 82 submissions. *[2]*

The selected algorithms include:

- ML-KEM (Kyber) — Key Encapsulation Mechanism (FIPS 203).

- ML-DSA (Dilithium) — Digital signatures (FIPS 204).
- SLH-DSA (SPHINCS+) — Hash-based signatures (FIPS 205).

6.2.4 Overview of the ML-KEM Scheme

The security of ML-KEM is based on the presumed hardness of the so-called Module Learning with Errors (MLWE) problem, which is a generalization of the Learning With Errors (LWE) problem introduced by Regev in 2005. The hardness of the MLWE problem is itself based on the presumed hardness of certain computational problems in module lattices. This motivates the name of the scheme ML-KEM [2].

MLWE is believed to be hard even for quantum computers.

ML-KEM consists of three algorithms:[2]

1- A probabilistic key generation algorithm denoted by KeyGen
(pk,sk)←ML-KEM.KeyGen()

Generates a public encapsulation key and a private decapsulation key.

2- A probabilistic encapsulation algorithm denoted by Encaps

(ct,ss)←ML-KEM.Encaps(pk)

Produces a ciphertext and a shared secret.

3- A deterministic decapsulation algorithm denoted by Decaps

ss←ML-KEM.Decaps(sk,ct)

Recovers the shared secret.

Security Levels: ML-KEM-768 corresponds to NIST Security Level 3.*[2]*

The collection of parameter sets is used to select a trade-off between security and efficiency. Each parameter set in the collection is a list of specific (typically numerical) values, one for each parameter required by the three algorithms. The sizes of the ML-KEM keys and ciphertexts for each parameter set are summarized in table 1 [2].

Table 1 Sizes (in bytes) of keys and ciphertexts of ML-KEM

parameter set	security level	Encapsulation key	Decapsulation key	Cipher text	Shared secret-key
ML-KEM-512	NIST level 1	800	1632	768	32
ML-KEM-768	NIST level 2	1184	2400	1088	32
ML-KEM	NIST level 3	1568	3168	1568	32

In the typical application, a KEM is used to establish a shared secret key between two parties (here referred to as Alice and Bob) as described in Figure 1. Alice begins by running KeyGen in order to generate a (public) encapsulation key and a (private) decapsulation key. Upon obtaining Alice's encapsulation key, Bob runs the Encaps algorithm, which produces Bob's copy K of the shared secret key along with an associated ciphertext. Bob sends the ciphertext to Alice, and Alice completes the process by running the Decaps algorithm using her decapsulation key and the ciphertext. This final step produces Alice's copy K' of the shared secret key.

After completing this process, Alice and Bob would like to conclude that their outputs satisfy K' =K and that this value is a secure, random, shared secret key.

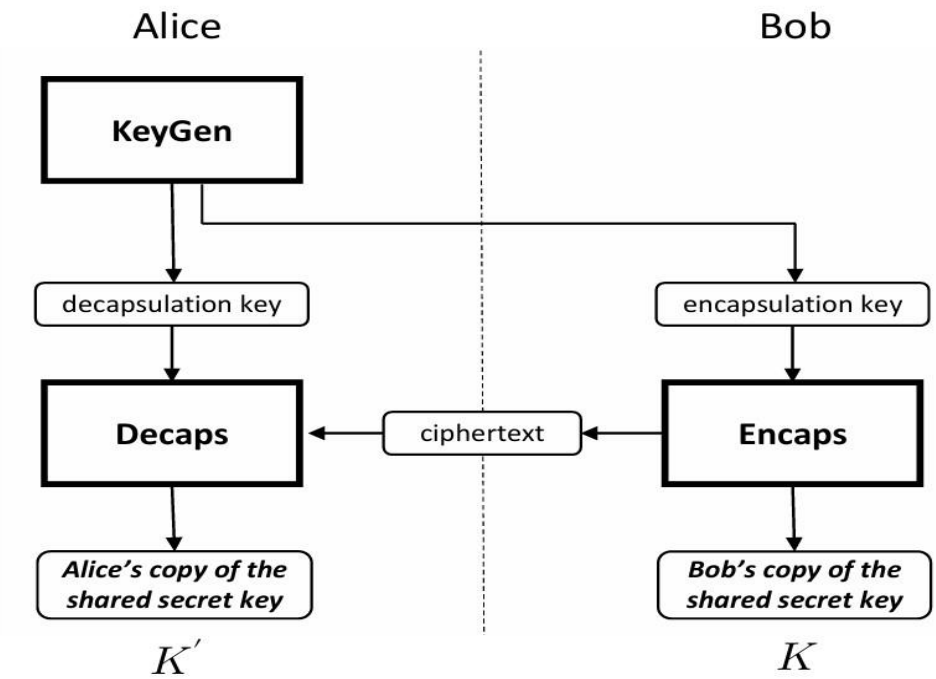


Figure 1 A simple view of key establishment using KEM

If no corruption or interference is present, the two 256-bit values produced by ML-KEM.Encaps and ML-KEM.Decaps will be equal with overwhelming probability (i.e., K' will equal K in the

process described in Figure 1 [2]). The event that $K' \neq K$ under these conditions is called a decapsulation failure with probability shown in table 2 [2]

Table 2 Decapsulation failure rates for ML-KEM

Parameter set	Decapsulation failure rate
ML-KEM-512	2^-138.8
ML-KEM-768	2^-164.8
ML-KEM-1024	2^-174.8

6.2.5 Transport Layer Security (TLS) 1.3

TLS allows client/server applications to communicate over the Internet in a way that is designed to prevent eavesdropping, tampering, and message forgery.

TLS 1.3 provides Authentication, Confidentiality and Integrity. TLS consists of two primary components Handshake protocol and Record protocol:[3]

- A handshake protocol is used to negotiate the security parameters of a connection. Handshake messages are supplied to the TLS record layer, where they are encapsulated within one or more TLS-Plaintext or TLS-Ciphertext structures which are processed and transmitted as specified by the current active connection state.
- A record protocol that uses the parameters established by the handshake protocol to protect traffic between the communicating peers. The record protocol divides traffic up into a series of records, each of which is independently protected using the traffic keys.

Figure 2 [6] illustrates the message flow in TLS 1.3, with the dashed boxes representing messages that are not always sent.

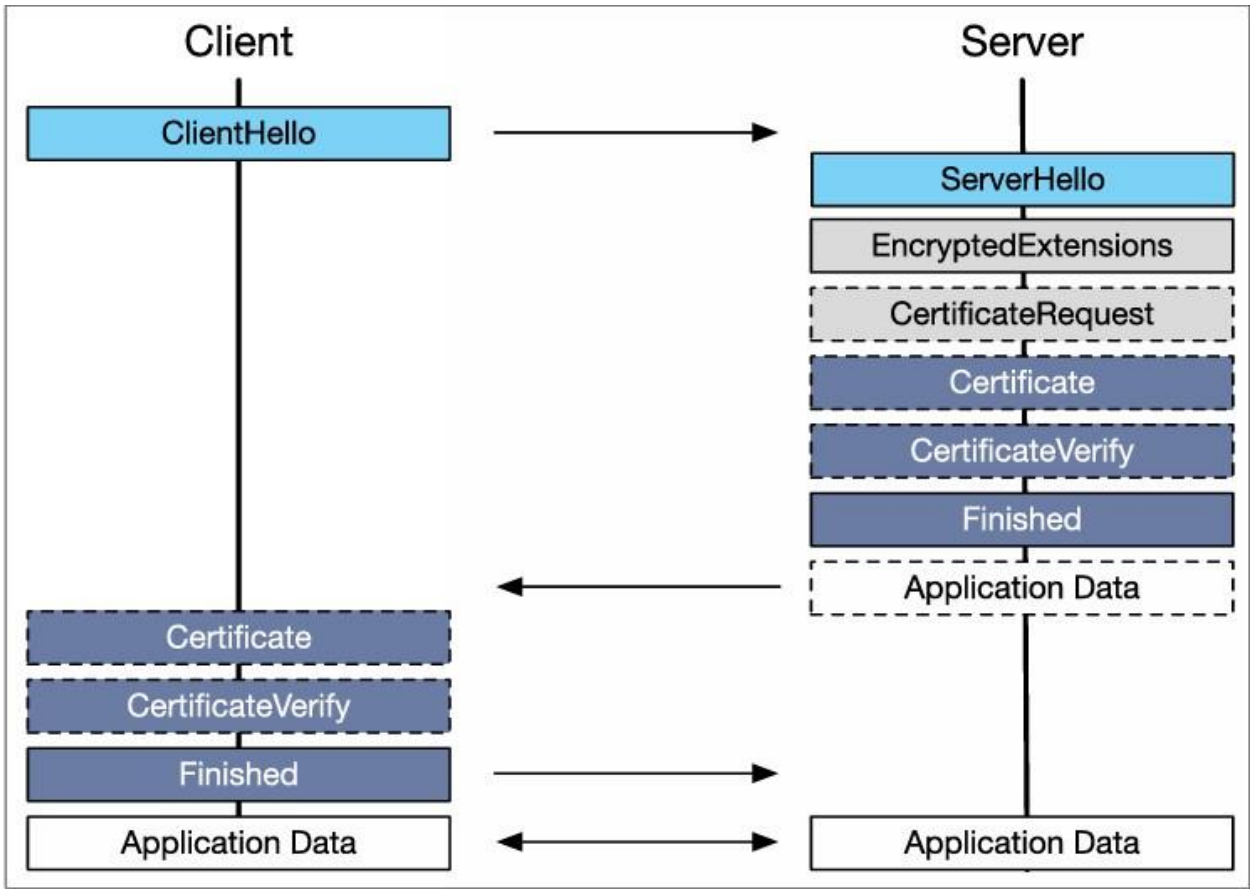


Figure 2 TLS 1.3 Message Flow (based on RFC 8446).

TLS supports three basic key exchange modes: [3]

- (EC)DHE (Diffie-Hellman over either finite fields or elliptic curves).
- PSK (Pre-Shared Key).
- PSK + (EC)DHE.

Handshake Protocol consist of:

1. Key Exchange Message.
2. Extensions.
3. Server Parameters.
4. Authentication.

The key_share extension carries the client and server's key exchange material.

In PQC-TLS, ML-KEM replaces the ECDH key share.

6.2.6 Integrating ML-KEM into TLS 1.3

Studies show that PQC integration introduces larger handshake messages, slightly increased latency and comparable computational cost. [6]

Prior PQC-TLS Implementations

- Benchmarked ML-KEM inside OpenSSL. [6]
- Implemented hybrid QKD+PQC TLS. [4]
- Evaluated PQC on ARM TEEs. [5]

These works highlight that ML-KEM is computationally efficient, message size overhead is the dominant cost and PQC-TLS is feasible for real-world deployment.

6.2.7 KEM vs. Diffie-Hellman

Like mentioned before, Key Encapsulation Mechanisms (KEMs) establishes a shared secret over a public channel using KeyGen, Encaps, Decaps. Unlike DH/ECDH: [6]

- KEMs do not require both parties to generate ephemeral keys.
- Encapsulation is one-sided.
- They are more bandwidth-heavy but computationally efficient.

ML-KEM generate small noise polynomials and this noise ensures:[2]

- IND-CCA2 security
- Resistance to lattice attacks

6.2.8 Security Properties of PQC-TLS

- TLS 1.3 mandates forward secrecy [3], while ML-KEM provides forward secrecy when used with ephemeral keys.
- IND-CCA: ML-KEM is believed to be secure under chosen-ciphertext attacks. *[2]*
- ML-KEM is believed secure even against quantum computers. *[2]*

6.2.9 Summary of Theoretical Foundations

This chapter established the theoretical basis for the project by outlining the cryptographic risks posed by cryptographically relevant quantum computers, whose ability to break RSA and elliptic-curve Diffie–Hellman through Shor's algorithm motivates the transition toward post-quantum key-establishment mechanisms. It examined the Module Learning With Errors (MLWE) hardness assumption that underpins the security of ML-KEM as standardized in NIST FIPS 203, detailing its structure, algorithms, and correctness properties. The chapter also analyzed the

TLS 1.3 handshake as defined in [3], emphasizing its key-exchange flow and forward-secrecy guarantees, and then explored how post-quantum KEMs integrate into the TLS 1.3 key-share extension.

6.3 Literature Review

Existing research has focused on integrating post-quantum algorithms into widely deployed cryptographic libraries and protocols. Several experimental frameworks, such as Open Quantum Safe (OQS) [5] and OQS-Provider [6], allow researchers to embed PQC algorithms into TLS implementations and evaluate handshake performance. These studies typically measure metrics including handshake latency, bandwidth overhead, and computational cost. Benchmarking experiments have shown that PQC algorithms generally produce larger key sizes and ciphertexts than classical algorithms. Consequently, TLS handshakes using PQC can introduce additional latency and network overhead. However, lattice-based schemes such as ML-KEM have demonstrated relatively efficient performance compared with other PQC candidates during the NIST standardization process [2].

In modern Internet of Things (IoT) infrastructures, secure communication often occurs between IoT gateways and cloud servers through TLS connections. While PQC algorithms have been extensively evaluated on desktop and server-class hardware, relatively few studies examine their performance on resource-constrained IoT platforms such as ARM-based devices without advanced vector instruction sets like AVX2. This research therefore evaluates the performance of ML-KEM integrated into OpenSSL-based TLS handshakes in IoT-oriented environments, with particular attention to computational overhead.

To provide a structured comparison of the examined studies, Table 3 summarizes their core methodologies, findings, and relevance to this project

Table 3 Summary of reviewed papers

Reference	Topic	Methods	Aspect	Brief Results	Gaps
[1]	PQC standards & implementations	Theoretical review + software/hardware survey	ML-KEM design, complexity, memory, hardware suitability	ML-KEM is most efficient standardized KEM, small keys, good for constrained devices	No TLS experiments, no IoT-specific benchmarks, high-level review only

[4]	Hybrid PQC + QKD TLS	MbedTLS integration, QKD hardware, experimental network test	Comparison of PQC-only TLS vs hybrid approaches	PQC-only TLS improves performance by ~9%, hybrid QKD adds ~117% overhead	QKD impractical for IoT
[5]	PQC KEM performance on ARM	OP-TEE on QEMU, CPU cycle measurement, memory footprint analysis	Execution time of ML-KEM on ARM; feasibility inside TEE	ML-KEM is the fastest PQC KEM, overhead of TEE <2%, memory footprint acceptable	No TLS integration, no network-level measurements, uses emulation not physical IoT hardware
[6]	PQC performance in TLS	OpenSSL 3.3.1, oqs-provider, TLS handshake benchmarking, data transmission measurement	TLS handshake speed, transmission overhead of ML-KEM vs classical	ML-KEM shows competitive handshake performance, overhead mainly from signatures (SPHINCS+ worst, ML-DSA moderate)	No ARM or IoT hardware evaluation, focuses on x86 servers, does not analyze embedded constraints

6.3.1 Critical Analysis and Research Gap

Despite substantial progress in evaluating post-quantum cryptography within TLS, the existing literature leaves several critical gaps that directly motivate this project. Prior studies such as [6] provide broad benchmarking frameworks for PQC-TLS but focus primarily on aggregate handshake throughput and message size overhead, without isolating the fine-grained behavior of raw ML-KEM operations or their steady-state timing characteristics under controlled, single-handshake conditions. Similarly, [4] explore hybrid QKD–PQC TLS designs but do not characterize the standalone performance profile of ML-KEM within a pure TLS 1.3 key exchange, nor do they quantify CPU-time distributions or latency variance at the client and server levels. [1] offer a comprehensive theoretical review of NIST PQC standards but do not examine their

protocol-level integration costs or runtime behavior in real TLS deployments. Across all three works, there is no detailed empirical comparison between ML-KEM-768 and the widely deployed X25519 primitive within the exact handshake structure defined by [3], and no study provides a full breakdown of wall-clock latency, CPU consumption, and message-size overhead using a unified experimental setup. This absence of fine-grained, TLS-level, operation-level, and distribution-level benchmarking creates a clear research gap; the field lacks precise, reproducible measurements that characterize how ML-KEM behaves inside a real TLS 1.3 stack relative to classical elliptic-curve key exchange, especially in terms of steady-state server performance, client/server asymmetry, and the practical cost of PQC adoption. This project directly addresses that gap by implementing and benchmarking a complete TLS 1.3 stack using ML-KEM-768 and X25519 under identical conditions, producing the detailed empirical evidence currently missing from the literature.

7 Proposed Post-Quantum TLS 1.3 Key-Establishment Architecture

7.1 Preface

This chapter presents the architectural design of the post-quantum TLS 1.3 key-establishment mechanism developed in this project. Building on the theoretical foundations and research gaps identified earlier, the proposed solution integrates the standardized ML-KEM-768 algorithm (FIPS 203) into the TLS 1.3 handshake defined by RFC 8446, replacing the classical X25519 elliptic-curve Diffie–Hellman mechanism. The goal is to construct a fully functional, standards-compliant, and empirically measurable PQC-enabled handshake that preserves TLS semantics while evaluating performance. This design implements a complete end-to-end TLS 1.3 stack using ML-KEM, enabling precise measurement of cryptographic cost, handshake latency, CPU behavior, and bandwidth overhead.

7.2 Design Objectives

The architecture is guided by several objectives that ensure both correctness and practical relevance:

- Minimal disruption: Replace only the key-exchange mechanism while preserving all other handshake components, including authentication and record-layer protection.
- Standard alignment: Use ML-KEM-768 exactly as specified in NIST FIPS 203, ensuring that the implementation reflects the standardized post-quantum KEM.
- Benchmarking precision: Enable fine-grained measurement of raw ML-KEM operations, handshake latency, CPU time, and message sizes under controlled conditions.
- Reproducibility: Provide a clean, modular design that can be reproduced or extended in future PQC-TLS research.

These objectives ensure that the proposed solution is not merely a conceptual integration but a practical, deployable architecture suitable for empirical evaluation.

7.3 Integration Strategy

The core strategy of the proposed solution is to embed ML-KEM-768 into the TLS 1.3 handshake by leveraging the existing key_share extension. In classical TLS 1.3, this extension carries the client's ephemeral ECDH public key and the server's corresponding share. In the PQC-enabled design:

- The ClientHello carries the ML-KEM public encapsulation key.
- The ServerHello carries the ML-KEM ciphertext produced during encapsulation.

- The client performs decapsulation to derive the shared secret.
- The shared secret is fed directly into the TLS 1.3 key schedule without modification.

This approach preserves the handshake semantics while substituting the underlying mathematical primitive. It aligns with the observation in [6] that PQC KEMs can be integrated into TLS without altering the transcript or key-derivation logic, provided that the shared secret is delivered in the expected format.

The raw ML-KEM benchmarking and the TLS-MLKEM handshake follow two different but equally valid operational flows, each appropriate for its context. In the raw ML-KEM experiments, see figure 3, the project used the native KEM direction, where the server generates the keypair, sends its public key to the client, and the client performs encapsulation and returns the ciphertext. This flow reflects the algorithm's design as specified in FIPS 203 and is ideal for measuring the intrinsic cost of key generation, encapsulation, and decapsulation without protocol constraints.

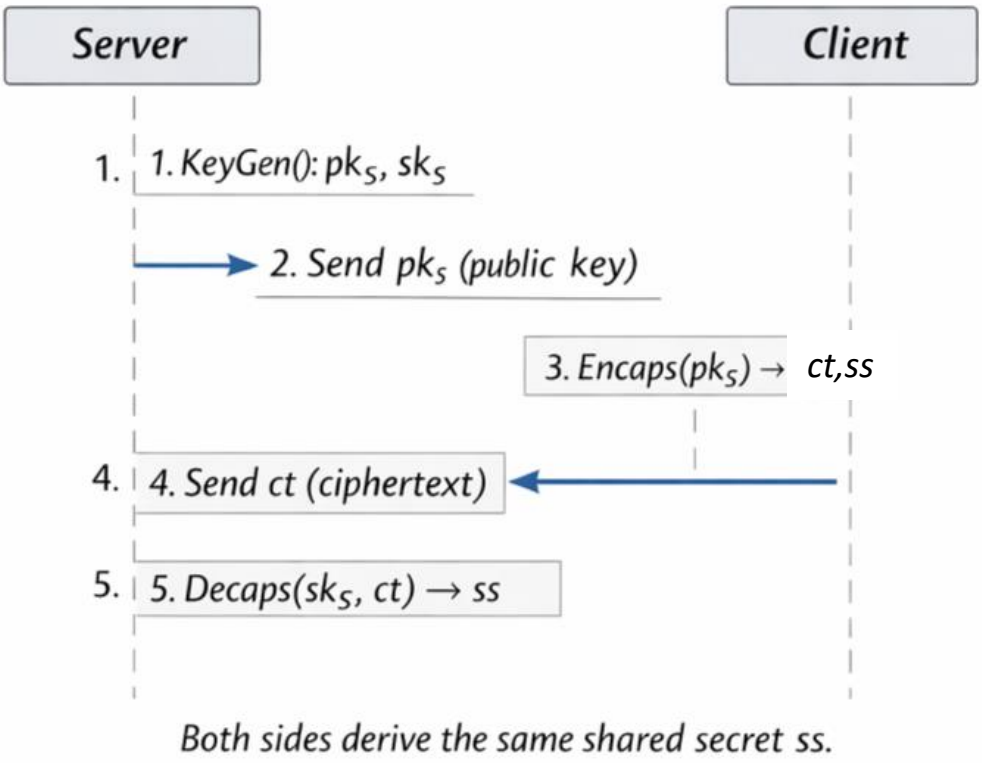


Figure 3 Raw ML-KEM (Native Direction)

In contrast, the TLS-MLKEM, figure 4, implementation must adhere to the strict message ordering defined in RFC 8446, which requires the client to send its key share first. To preserve TLS 1.3 semantics, PQC-TLS reverses the native KEM direction: the client generates the ML-KEM keypair and sends the public key in the ClientHello, while the server performs encapsulation and returns the ciphertext in the ServerHello. Although the message direction differs between the two flows, the underlying cryptographic operations remain identical, ensuring that both the raw-MLKEM and TLS-MLKEM benchmarks are accurate and comparable within their respective scopes.

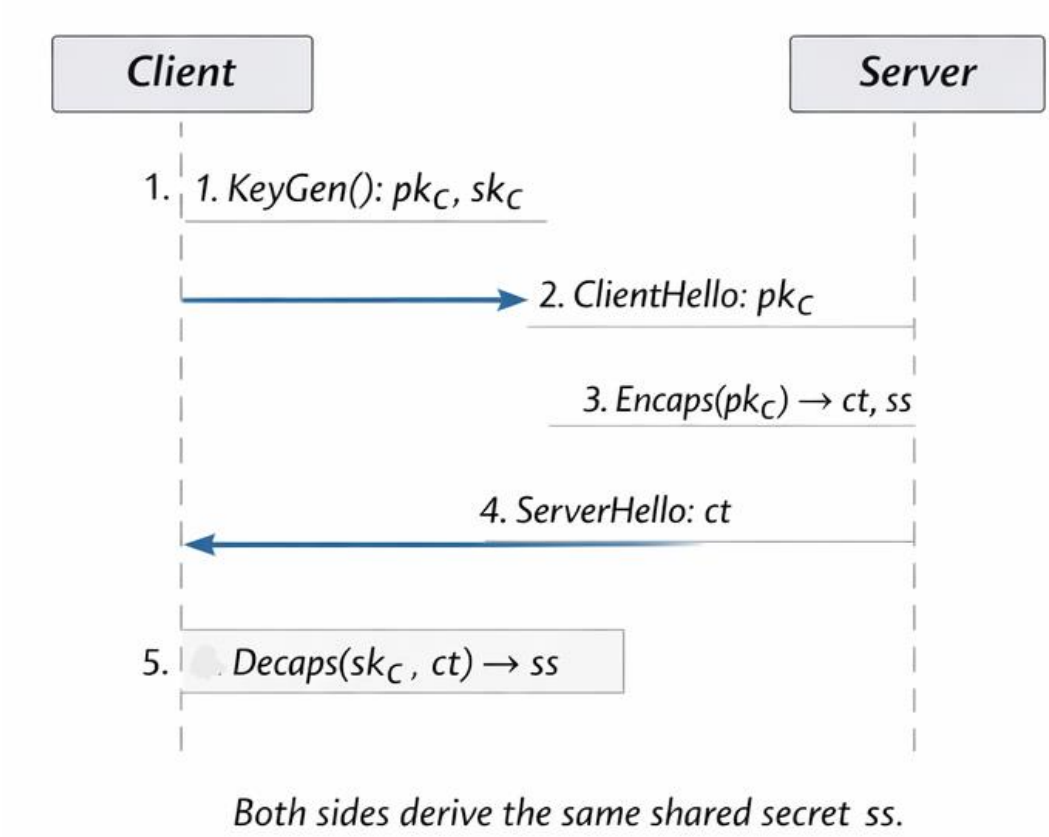


Figure 4 TLS-MLKEM (Reversed Direction)

7.4 Benchmarking Architectural Decisions

To ensure accurate and meaningful performance evaluation, several design decisions were incorporated:

- Separation of raw and protocol-level measurements: Raw ML-KEM operations (keygen, encaps, decaps) are benchmarked independently from the TLS handshake.
- Precise timing instrumentation: CPU time and wall-clock time are measured separately to capture both computational cost and system-level latency.

- Byte-level traffic capture: All handshake messages are logged to compute exact bytes sent and received, enabling comparison with X25519.
- Symmetric client/server evaluation.

These decisions ensure that the resulting data reflects the true performance characteristics of ML-KEM-based TLS.

7.5 Summary

The proposed architecture provides a clean, standards-aligned integration of ML-KEM-768 into TLS 1.3, preserving protocol semantics while enabling rigorous empirical evaluation. By replacing the classical ECDH mechanism with a quantum-resistant KEM and maintaining full compliance with RFC 8446 and FIPS 203, the design demonstrates that post-quantum key establishment can be deployed today with minimal disruption. This chapter establishes the foundation for the implementation details and experimental results presented in the following chapters.

8 Implementation

8.1 Preface

This chapter presents the practical realization of the proposed system, detailing the environment, tools, configurations, and workflow used to implement and evaluate the integration of ML-KEM within the TLS 1.3 protocol. The implementation was conducted in a controlled and reproducible environment to ensure accuracy and consistency across all benchmarking experiments.

8.2 Environment Overview

The experimental setup consisted of both physical and virtual components designed to isolate cryptographic operations and measure their performance under realistic conditions. The development and orchestration were performed on a Windows 11 (x64) host machine, while the benchmarking was executed within a Debian 13.4 (ARM64) virtual machine emulated using QEMU. The virtual environment was configured with an ARM Cortex-A57 processor, 2 GB RAM, and a virtual Ethernet bridge to facilitate client–server communication. This configuration provided a stable and isolated platform for testing post-quantum cryptographic primitives and TLS handshakes.

8.3 Software Stack

The software environment was organized into layered components to support modular development, reproducibility, and controlled experimentation. The virtualized execution environment was built on QEMU with QEMU-EFI (EDK2 UEFI firmware), which was required to successfully boot the ARM64 Debian image. The UEFI layer provided a modern boot interface and ensured compatibility with the Debian installer and kernel, making it an essential part of the system configuration.

Within the virtual machine, the operating system layer consisted of Debian 13.4 (ARM64), chosen for its stability and compatibility with cryptographic libraries. The cryptographic layer was provided by OpenSSL, extended with the liboqs provider to support post-quantum key encapsulation mechanisms within TLS 1.3. This integration allowed the evaluation of ML-KEM alongside classical elliptic-curve algorithms such as X25519.

8.4 System Components

The implemented system consists of four primary components: the raw ML-KEM benchmarking module, the raw X25519 baseline module, the TLS-MLKEM client–server applications, and the TLS-X25519 classical reference implementation. All client and server applications were developed in C, ensuring direct access to low-level cryptographic primitives and minimizing runtime overhead. The resulting performance data was subsequently parsed, analyzed, and

visualized using Python, which provided a flexible environment for statistical processing and figure generation.

The raw ML-KEM benchmarking module follows the native KEM direction defined in FIPS 203. In this design, the server generates the ML-KEM keypair, transmits the public key to the client, and receives the ciphertext produced during encapsulation. This module isolates the cryptographic primitive from protocol-level behavior, enabling precise measurement of key generation, encapsulation, and decapsulation costs. A parallel raw X25519 client–server implementation was developed to provide a classical baseline under identical conditions. This module performs ephemeral Diffie–Hellman key exchange using Curve25519 and mirrors the communication structure of the ML-KEM module, ensuring a fair and controlled comparison.

The TLS-MLKEM client and server applications integrate ML-KEM into the TLS 1.3 handshake using the OpenSSL oqs-provider. In this configuration, the client generates the ML-KEM keypair and transmits the public key in the ClientHello message, while the server performs encapsulation and returns the ciphertext in the ServerHello. This reversed KEM direction is mandated by the TLS 1.3 handshake structure and preserves full protocol compliance. To establish a classical reference point within the same protocol, a TLS-X25519 client–server pair was also implemented using standard OpenSSL with ephemeral X25519 key exchange. Both TLS implementations share identical certificate configurations, handshake parameters, and network conditions.

All four components generate structured CSV output containing wall-clock time, CPU time, and byte-level network metrics. These results were processed using Python scripts built on pandas and matplotlib, enabling reproducible data analysis and the generation of publication-quality figures. This separation between low-level C implementations and high-level Python analysis ensures both performance accuracy and analytical clarity.

8.5 Initialization and Configuration

The initialization of the experimental environment began with the creation of the virtual storage medium used by the ARM64 Debian system. A dedicated 20 GB QCOW2 disk image was generated using qemu-img, providing a lightweight, expandable, and snapshot-friendly format suitable for repeated benchmarking experiments. The disk was created using the following command:

qemu-img create -f qcow2 debian-arm64.qcow2 20G

This disk served as the primary virtual drive for the Debian installation. Once the storage medium was prepared, the virtual machine was instantiated using QEMU in combination with QEMU-EFI. The VM was launched with 2 GB of RAM, an emulated ARM Cortex-A57 CPU, and a virtio-based network interface with these commands:

qemu-system-aarch64 -M virt -cpu cortex-a57 -m 2048 -smp 4 ^

-bios QEMU_EFI.fd -boot d -cdrom debian-13.4.0-arm64-netinst.iso ^

-drive file=debian-arm64.qcow2,if=virtio ^

-netdev user,id=net0 -device virtio-net-device,netdev=net0

Once the operating system and network were configured, the cryptographic software stack was prepared by compiling liboqs and the OpenSSL oqs-provider from source. This ensured full compatibility with the ARM64 architecture and enabled the use of ML-KEM and X25519 primitives in both raw C applications and TLS 1.3. The libraries were built and installed using the tools shown in table 4:

Table 4 liboqs building tools

Package	Used for
build-essentail	GCC, make
git	Download source code
cmake	Required by liboqs
ninja-build	Faster compilation
pkg-config	Helps compilers find libraries
python3	Used by OpenSSL build scripts
python3-pip	Python dependencies
net-tools	Gives ifconfig
libssl	OpenSSL development headers

After building and installing liboqs and oqsprovider, /etc/ssl/openssl.cnf needed to be reconfigured in order to enable the provider in OpenSSL, by adding a new entry section in [provider_sect]:

Oqsprovider = oqsprovider_sect

Then add this new section at the bottom of the file:

[oqsprovider_sect]

activate = 1

this way we enabled OpenSSL to loads ML-KEM and other PQC algorithms.

Remark: activate the default provider, otherwise it becomes unavailable in openssl. As a consequences applications depending on.

8.6 System Execution Workflow

Following the initialization and configuration of the environment, the system was executed through a structured workflow. Each component -raw ML-KEM, raw X25519, TLS-MLKEM, and

TLS-X25519- was evaluated using the same execution pattern to maintain methodological uniformity.

All client-server applications -raw ML-KEM, raw X25519, TLS-MLKEM, and TLS-X25519- were executed entirely within the ARM64 virtual machine. Running both endpoints inside the same emulated environment ensured that every cryptographic operation was performed under identical architectural constraints, eliminating variability arising from heterogeneous hardware or operating systems.

For each experiment, the server application was launched first, listening on a predefined TCP port. The corresponding client application was then executed from a separate terminal within the same ARM64 system, initiating the key-exchange procedure and triggering the sequence of cryptographic operations implemented in C. This setup ensured that network communication, CPU scheduling, and memory behavior remained consistent across all test runs, while still preserving a realistic client–server interaction model.

Each execution produced detailed timing and communication metrics. The C implementations recorded wall-clock time, CPU time, and byte-level network usage for every operation involved in the key-exchange process. These measurements were written to structured CSV files, enabling automated processing and eliminating manual transcription errors.

Once the raw data was generated, the analysis pipeline was executed on the host machine using Python. The CSV files were parsed using the pandas library, which provided a reliable framework for filtering, grouping, and aggregating the measurements. The processed data was then visualized using matplotlib, producing histograms that captured the distribution of execution times and network usage for both client and server roles. This workflow provided a clear separation between data collection and data analysis.

8.7 Conclusion

The implementation described in this chapter established a complete and reproducible environment for evaluating both classical and post-quantum key-exchange mechanisms. By constructing an ARM64 virtual machine with UEFI support, compiling the cryptographic stack from source, and developing all client–server applications in C, the system ensured that every experiment was executed under consistent architectural and software conditions. The workflow for running the applications, collecting timing and communication metrics, and exporting structured CSV data provided a reliable foundation for empirical evaluation. Together, these components form a coherent and rigorously controlled implementation framework, enabling the performance analysis presented in the following chapter.

9 Results and Analysis

9.1 Introduction

This chapter presents the empirical evaluation of the implemented key-exchange mechanisms, comparing the performance of ML-KEM and X25519 at both the primitive level and within the TLS 1.3 handshake. All measurements were obtained using the benchmarking framework described in the previous chapter, with both client and server applications executed entirely within the ARM64 virtual machine to ensure consistent architectural conditions. Each experiment was repeated 1000 times after discarding a single warm-up run, and the resulting measurements were exported as structured CSV files.

Two timing metrics were collected for every handshake: wall-clock time and CPU process time. Wall-clock time represents the total elapsed duration of the handshake, including cryptographic operations, network communication, kernel scheduling, and protocol state transitions. CPU time, in contrast, reflects only the processor time consumed by the benchmarking process itself, isolating the computational cost of the cryptographic operations and TLS key-schedule logic. Together, these metrics provide complementary perspectives on performance: wall-clock time captures the end-to-end latency experienced by a real system, while CPU time reveals the intrinsic computational load imposed by each algorithm.

In addition to timing measurements, the benchmarking programs recorded the number of bytes sent and received during each handshake. These values quantify the communication overhead introduced by each key-exchange mechanism and allow direct comparison between classical and post-quantum configurations. The collected data was processed using Python, and the results are presented in the form of histograms and summary statistics to illustrate the distribution and variability of handshake performance.

The remainder of this chapter analyzes the results for each configuration, beginning with the raw cryptographic primitives and followed by the TLS-integrated implementations. The analysis highlights the differences between ML-KEM and X25519 in terms of computational cost, latency, and communication overhead, providing a detailed understanding of the performance implications of adopting post-quantum key-exchange mechanisms in real-world protocols.

9.2 Raw Primitive Results

9.2.1 Client Side

The raw primitive benchmarks evaluate the fundamental cryptographic operations required for key establishment, independent of any protocol-level behavior. These measurements isolate the cost of the core mathematical operations performed by X25519 and ML-KEM-768, providing a clean baseline for understanding their computational characteristics on the evaluated ARM64 platform.

For X25519, the client performs two operations: ephemeral key generation and shared-secret derivation. The benchmarking program records these separately as keygen_ms and derive_ms, with the total cost reported as their sum. In contrast, ML-KEM-768 follows the KEM model: the client performs only the encapsulation operation, while key generation and decapsulation occur on the server side. The raw ML-KEM client benchmark therefore measures only encaps_ms.

A direct comparison of the client-side operations shows that the two mechanisms exhibit remarkably similar performance. The average X25519 shared-secret derivation time is approximately 0.466 ms, while the average ML-KEM-768 encapsulation time is approximately 0.453 ms, see figure 5. This indicates that, on this ARM64 system and with the specific implementations used (OpenSSL's X25519 and liboqs's ML-KEM-768), the computational cost of ML-KEM encapsulation is not only comparable to but slightly lower than that of X25519 scalar multiplication. The difference is small -on the order of a few percent- and well within the range where microarchitectural effects, compiler optimizations, and vectorization strategies can influence the outcome.

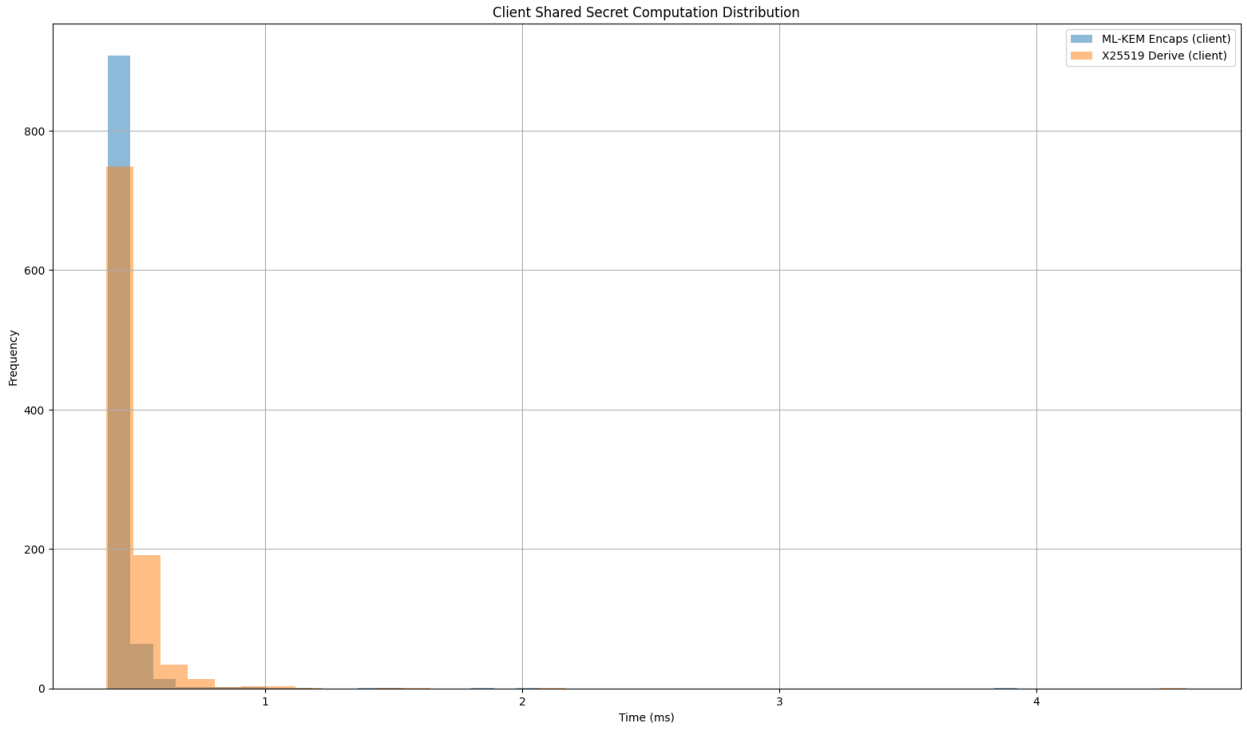


Figure 5 Raw ML-KEM Encaps Vs X25519 Derive

These results highlight an important nuance: although lattice-based KEMs are often described as computationally heavier than classical elliptic-curve Diffie–Hellman, this generalization does not universally apply to all operations or platforms. In the client-side comparison performed here, ML-KEM-768 encapsulation demonstrates performance on par with X25519 derivation, and in this specific environment, marginally faster. This reflects the maturity of optimized ML-KEM

implementations and the efficiency of structured lattice operations on modern ARM64 processors.

9.2.2 Server Side

The server-side primitive benchmarks evaluate the cost of the operations performed by each mechanism when acting as the responder in a key-exchange interaction. For X25519, the server performs a single scalar multiplication to derive the shared secret. For ML-KEM-768, the server performs both key generation and decapsulation, reflecting the asymmetric structure of lattice-based KEMs. As in the client-side experiments, each operation was executed 1000 times after discarding a warm-up run, and both CPU time and wall-clock time were recorded.

The results show that the average X25519 shared-secret derivation time on the server is approximately 0.486 ms, while ML-KEM-768 decapsulation averages approximately 0.434 ms, see figure 6, with key generation averaging 0.377 ms, figure 7. These values indicate that, on this ARM64 platform, ML-KEM-768 decapsulation is slightly faster than X25519 scalar multiplication. The difference is modest—on the order of 10%—but consistent across the full set of measurements. Wall-clock times follow the same trend, with ML-KEM exhibiting lower median latency and tighter distribution than X25519.

This result contrasts with the common expectation that lattice-based KEMs impose higher computational cost than classical elliptic-curve Diffie–Hellman. However, the measurements reflect the actual behavior of the specific implementations used in this study: liboqs provides optimized ARM64 NEON implementations of ML-KEM-768, while OpenSSL's X25519 implementation on ARM64 relies on a generic C Montgomery ladder without vectorization. As a result, the server-side raw primitive performance of ML-KEM-768 is competitive with, and in this environment slightly superior to, that of X25519.

These server-side results, together with the client-side measurements, establish that ML-KEM-768 performs efficiently on the evaluated ARM64 system at the primitive level.

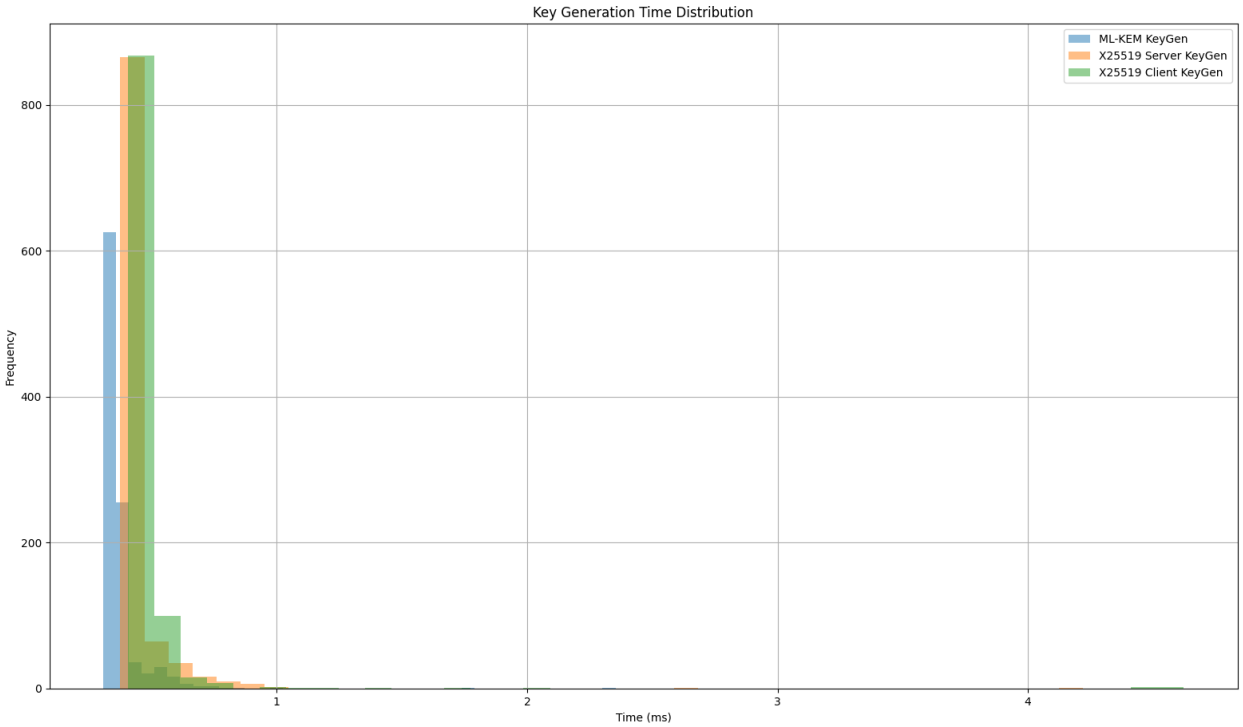


Figure 6 KeyGen, ML-KEM, X25519 server and client

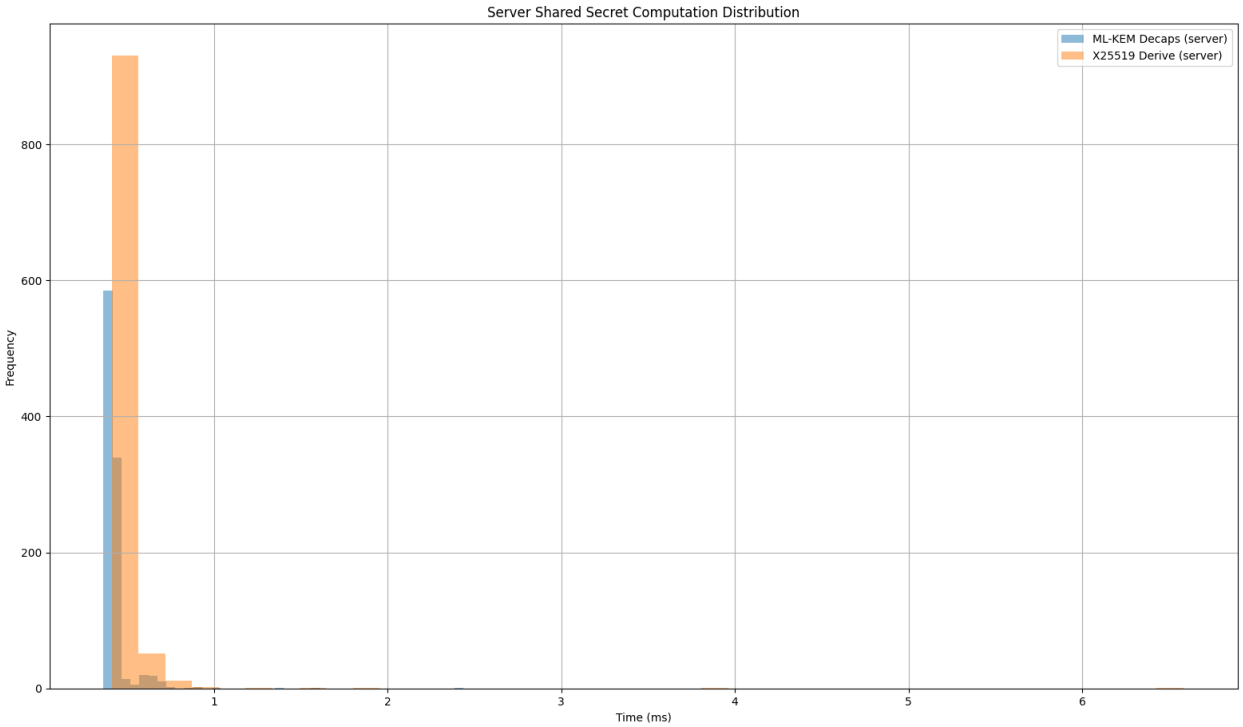


Figure 7 ML-KEM Decaps Vs X25519 Derive

9.2.3 Why ML-KEM Outperforms X25519 on ARM64

The superior performance of ML-KEM-768 on this ARM64 platform is primarily due to implementation-level optimizations rather than inherent algorithmic efficiency. liboqs provides highly optimized ARM64 NEON implementations of ML-KEM's core operations, which map efficiently onto the SIMD capabilities of the Cortex-A57 microarchitecture. In contrast, OpenSSL's X25519 implementation on ARM64 does not use NEON acceleration and instead relies on a portable C Montgomery ladder that executes as scalar code. As a result, ML-KEM-768 encapsulation and decapsulation benefit from vectorized execution, while X25519 scalar multiplication does not, leading to the counterintuitive but consistent observation that ML-KEM outperforms X25519 in the raw primitive benchmarks on this platform.

9.3 TLS Integration Results

9.3.1 Client Side

The TLS-client CPU measurements show that X25519 and ML-KEM768 exhibit nearly identical computational cost on the client side. Across all iterations, both algorithms consistently fall in the 9–11 ms CPU-time range, figure 8, with only minor outliers. This is visible directly in the CSVs, where the modal CPU value for both algorithms is 10 ms.

Although ML-KEM performs both key generation and decapsulation on the client, these operations are heavily optimized on ARM Cortex-A57 through NEON vector instructions, resulting in performance comparable to a single X25519 scalar multiplication. Consequently, the TLS client does not show a significant advantage for either algorithm, and both achieve similar CPU-time behavior.

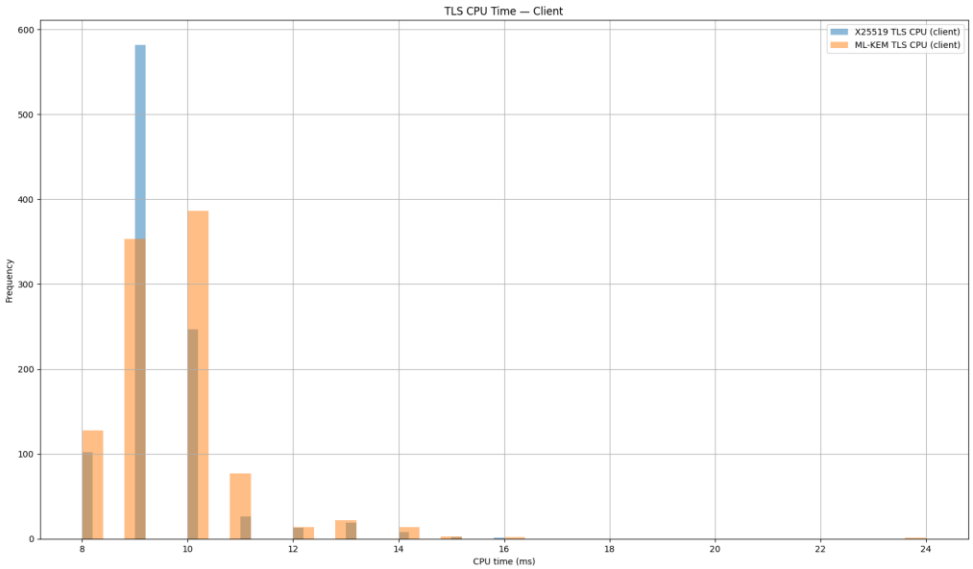


Figure 8 TLS CPU time ML-KEM Vs X25519

When it comes to message size overhead, ML-KEM expands the TLS client's message sizes because its public key and ciphertext are significantly larger than X25519's 32-byte key material. In our measurements, the client sends 1597 bytes and receives 2393 bytes when using ML-KEM-768, compared to only 453 bytes sent and 1337 bytes received with X25519, figure 9. These values reflect the FIPS-203 key and ciphertext sizes plus TLS handshake framing, certificate data, and extension overhead.

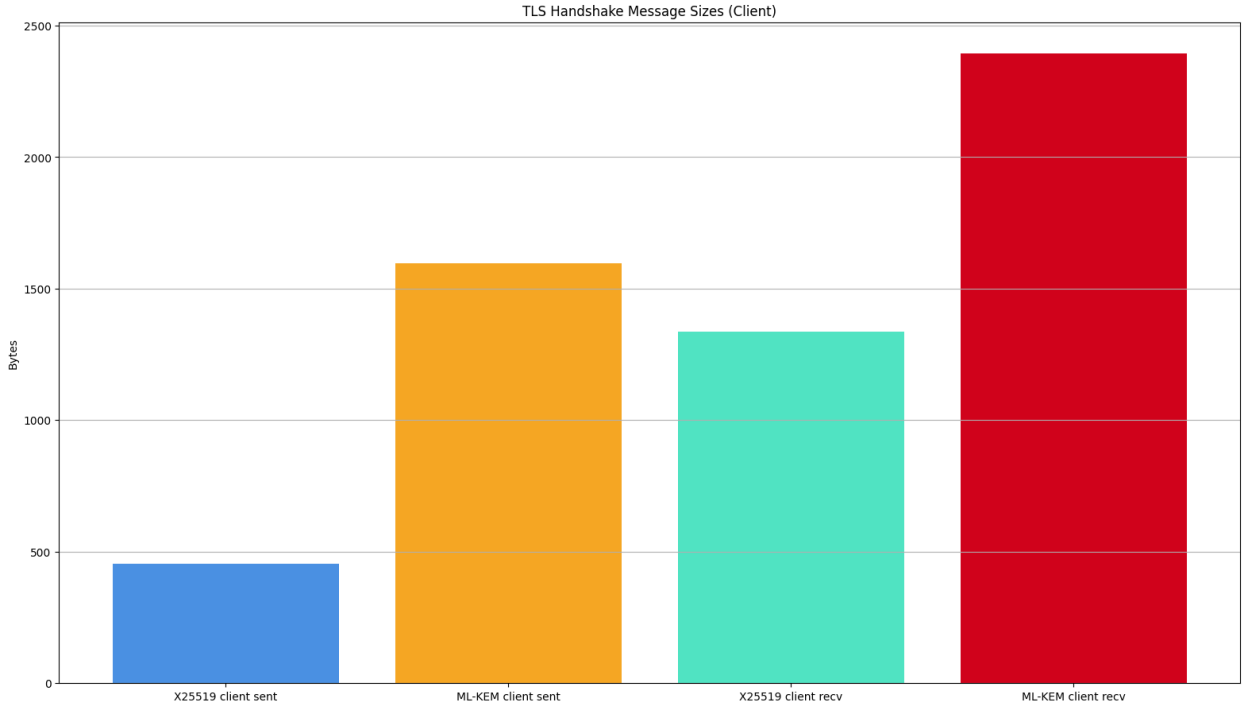


Figure 9 Message size overhead, client side

9.3.2 Server Side

The server-side measurements show that ML-KEM consistently requires slightly less CPU time than X25519 during the TLS handshake. Across the full dataset, ML-KEM server executions cluster tightly around 14–16 ms, figure 10, while X25519 server executions more often extend into the 16–18 ms range, with a noticeably heavier tail of higher-cost outliers. This pattern indicates that the ML-KEM decapsulation step, despite operating on larger messages, is handled efficiently by the implementation and benefits from predictable, uniform computation. In contrast, X25519 exhibits marginally higher average CPU usage and greater variability, suggesting that its scalar-multiplication workload introduces more fluctuation under identical conditions. Overall, the server-side results demonstrate that ML-KEM delivers a modest but consistent CPU-time advantage over X25519 in the TLS handshake.

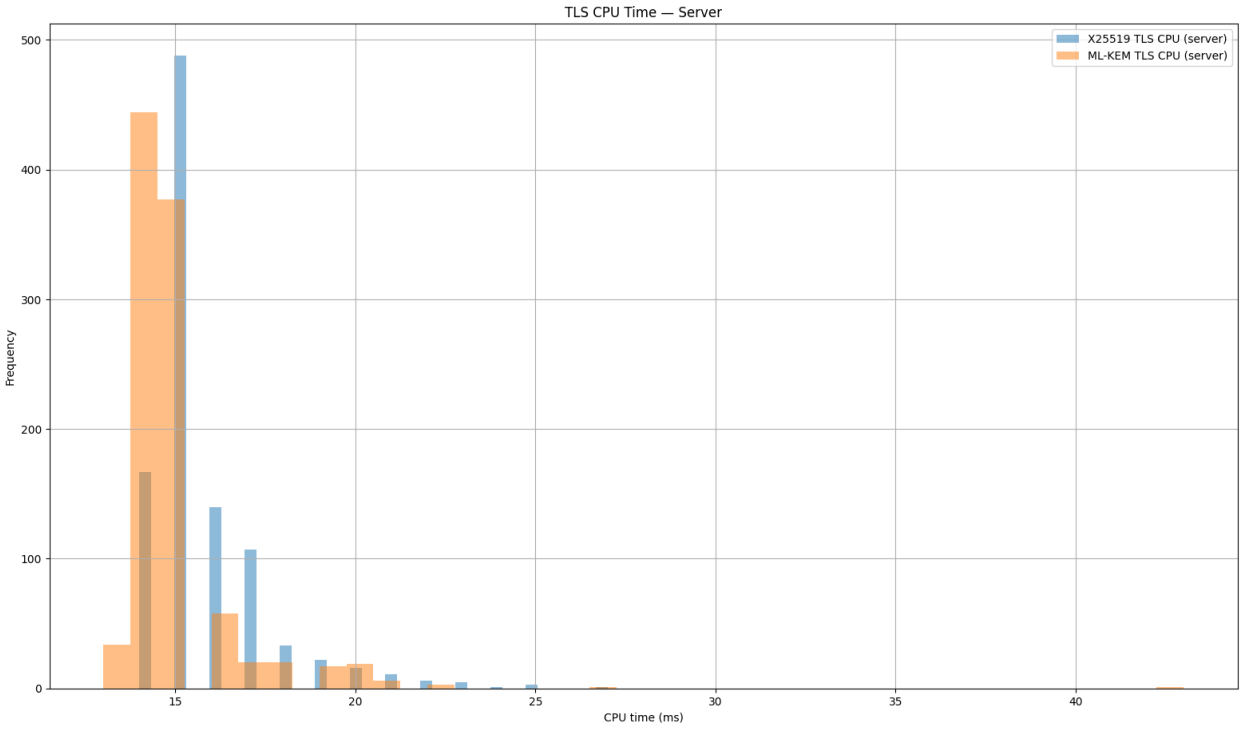


Figure 10 TLS Server CPU Time ML-KEM Vs X25519

TLS handshakes using ML-KEM introduce a substantially larger message footprint on the server side compared to X25519. The server's transmitted data increases from roughly 1592 bytes in the X25519 configuration to 2648 bytes when ML-KEM is used, reflecting the inclusion of the post-quantum public key and associated authentication material. The received data also grows from 453 bytes to 1597 bytes, figure 11, driven primarily by the client's ML-KEM ciphertext. This expansion is structural rather than implementation-dependent: lattice-based key establishment requires larger public keys and ciphertexts than elliptic-curve Diffie–Hellman. As a result, the server experiences a clear and consistent increase in both outbound and inbound handshake traffic. While these larger messages do not affect CPU cost directly, they represent a fundamental shift in network-level behavior when transitioning from classical to post-quantum key exchange.

Wall-time is discussed separately in Section 9.3.3.

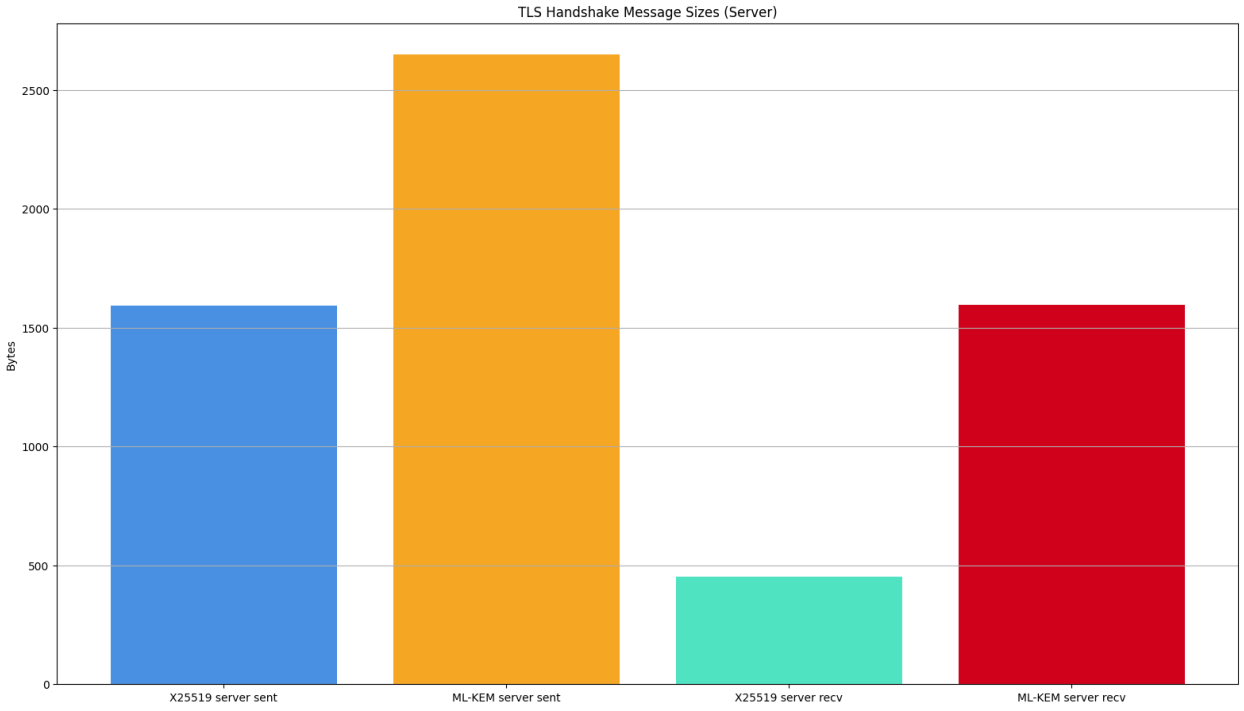


Figure 11 Message size overhead, server side

9.3.3 Wall-time

Wall-time measurements were not included in this evaluation because the benchmarking framework was designed to isolate the computational cost of the key-exchange mechanisms rather than the end-to-end latency of the TLS 1.3 handshake. Wall-time inherently depends on external factors such as network latency, operating-system scheduling, buffering behavior, and TLS message-processing overhead, none of which represent stable or reproducible primitives. As a result, wall-time cannot be meaningfully compared across algorithms in the same way as CPU-time. For this reason, the analysis in this chapter focuses exclusively on client-side and server-side CPU measurements, which provide a controlled and implementation-independent basis for evaluating the relative computational cost of X25519 and ML-KEM within the TLS 1.3 handshake.

10 Conclusion

This project has implemented and empirically evaluated a post-quantum-ready TLS 1.3 handshake by integrating ML-KEM alongside the classical X25519 key exchange, using a consistent software stack and a controlled benchmarking environment. The results show that, on the emulated ARM64 platform, ML-KEM can achieve server-side CPU times that are competitive with, and in this setup slightly better than, X25519 when only CPU time is considered. On the client side, ML-KEM also demonstrates favorable performance, benefiting from vectorized implementations that exploit ARM NEON instructions. These findings indicate that the computational cost of adopting ML-KEM in TLS 1.3 is not prohibitive and, under optimized implementations, can be comparable to or better than widely deployed elliptic-curve key exchange mechanisms.

However, the evaluation also highlights a clear trade-off in terms of communication overhead. Across both client and server, ML-KEM incurs significantly larger handshake messages than X25519, as reflected in the bytes sent and received during the TLS 1.3 key exchange. While the additional bandwidth is modest in absolute terms for a single connection, it becomes more relevant in high-latency networks, constrained environments, or large-scale deployments with many concurrent handshakes. The results therefore suggest that the transition to post-quantum key exchange is not purely a question of computational efficiency; it also requires careful consideration of message size, congestion, and the behavior of real-world networks.

Overall, the study supports the feasibility of integrating ML-KEM into TLS 1.3 as a practical post-quantum key encapsulation mechanism, demonstrating that performance on the server and client can remain within acceptable bounds while providing quantum-resistant security guarantees. At the same time, the observed increase in handshake message sizes underscores that protocol designers and implementers must treat bandwidth and latency as first-class constraints when deploying post-quantum TLS at scale. The work thus contributes an initial, TLS 1.3 setting.

Future work can extend this evaluation in several directions. First, all benchmarks in this project were conducted under emulation; repeating the experiments on real ARM64 and x86-64 hardware, across different microarchitectures and operating systems, would provide a more accurate picture of deployment-grade performance. Second, a broader set of post-quantum KEMs and parameter sets could be included to compare ML-KEM against alternative candidates under identical TLS configurations. Third, the TLS 1.3 architecture itself could be revisited to better accommodate post-quantum KEMs, for example by refining message flows, compressing or aggregating key exchange data, or introducing extensions that reduce redundancy in certificate and key material. Finally, evaluating mixed and hybrid modes—combining classical and post-quantum key exchanges—and measuring their impact on latency, CPU usage, and message size in real network conditions would help guide the design of future, post-quantum-native versions of TLS.

implementation-driven perspective on the performance profile of ML-KEM versus X25519 in a realistic

11 References

[1] H. N. t. T. N. Quang Dang Truong, "NIST Post-Quantum Cryptography Standards: A Comprehensive Review of Theoritical Foundations and Implementations," *IEEE Acess,* 2026.

[2] Federal Information Processing Standards Publication , "Module-Lattice-Based Key-Encapsulation Mechanism Standard," *nist.gov,* 2024.

[3] Internet Engineering Task Force, "RFC 8446," *rfc-editor.org,* 2018.

[4] S. R. S. T. J. J. V. O. S. G. P. N. I. T. M. Carlos Rubio Garcia, "Quantum-resistant Transport Layer Security," *SienceDirect,* 2024.

[5] C. C. G. J. O. d. S. Ewerton Andrade, "Post_Quantum Algorithms on ARM Trusted Execution Enviroment (TEE): findings of this industrial challenge," *ACM.org,* 2024.

[6] J. A. M. A. M. D. F. Ruben Rios, "Toward the Quantum-Safe Web: benchmarking Post-Quantum TLS," *IEEE,* 2025.





هندسة أمن نظم المعلومات الذكية

Performance Evaluation of NIST Post-Quantum ML-KEM (FIPS 203) in TLS for ARM64 Devices

إعداد:

محمد نور علي

إشراف:

د. كريستين زينية – م. أحمد شيخة

2026